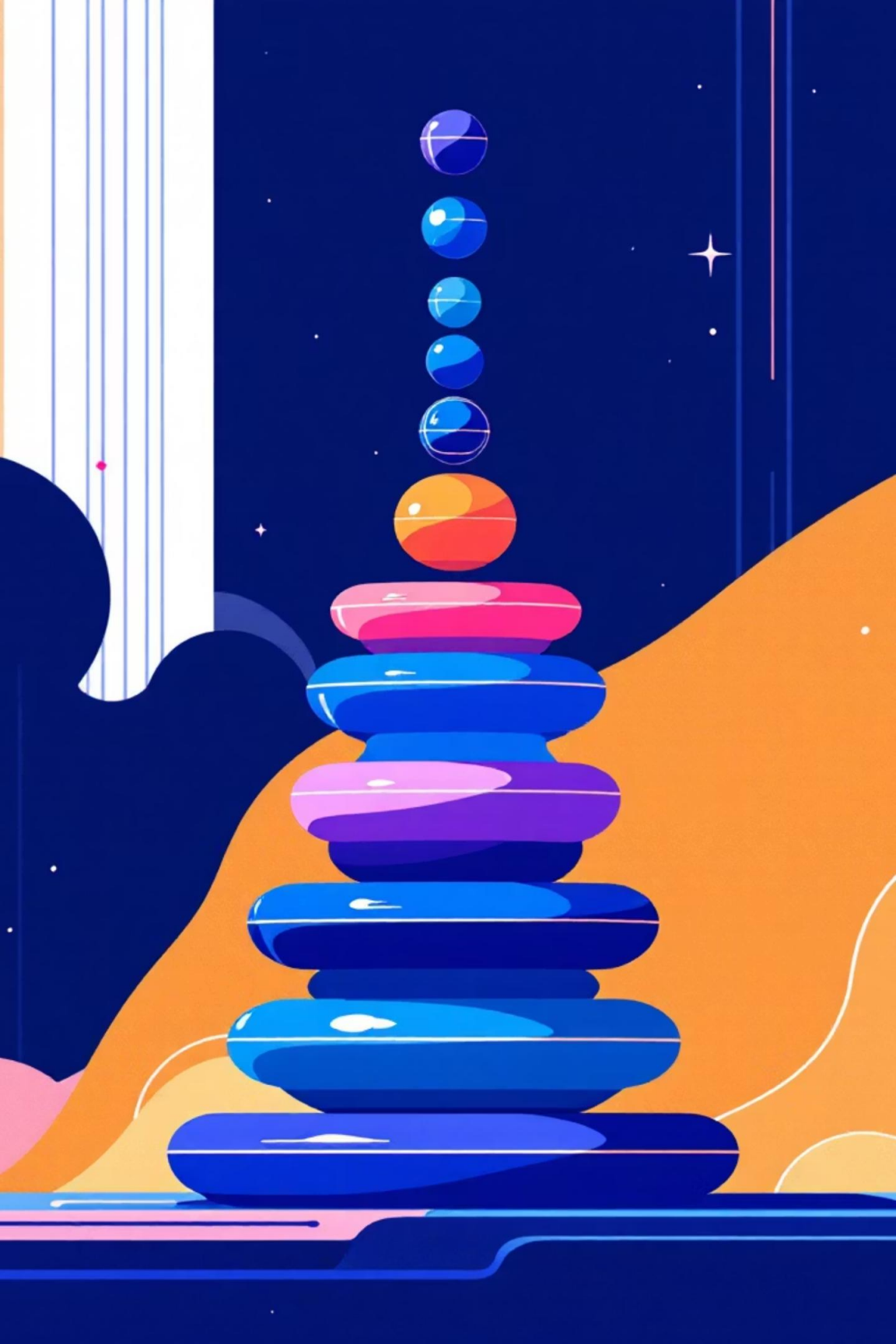




Tower of Hanoi

Mastering Recursion and Call Stack Behaviour

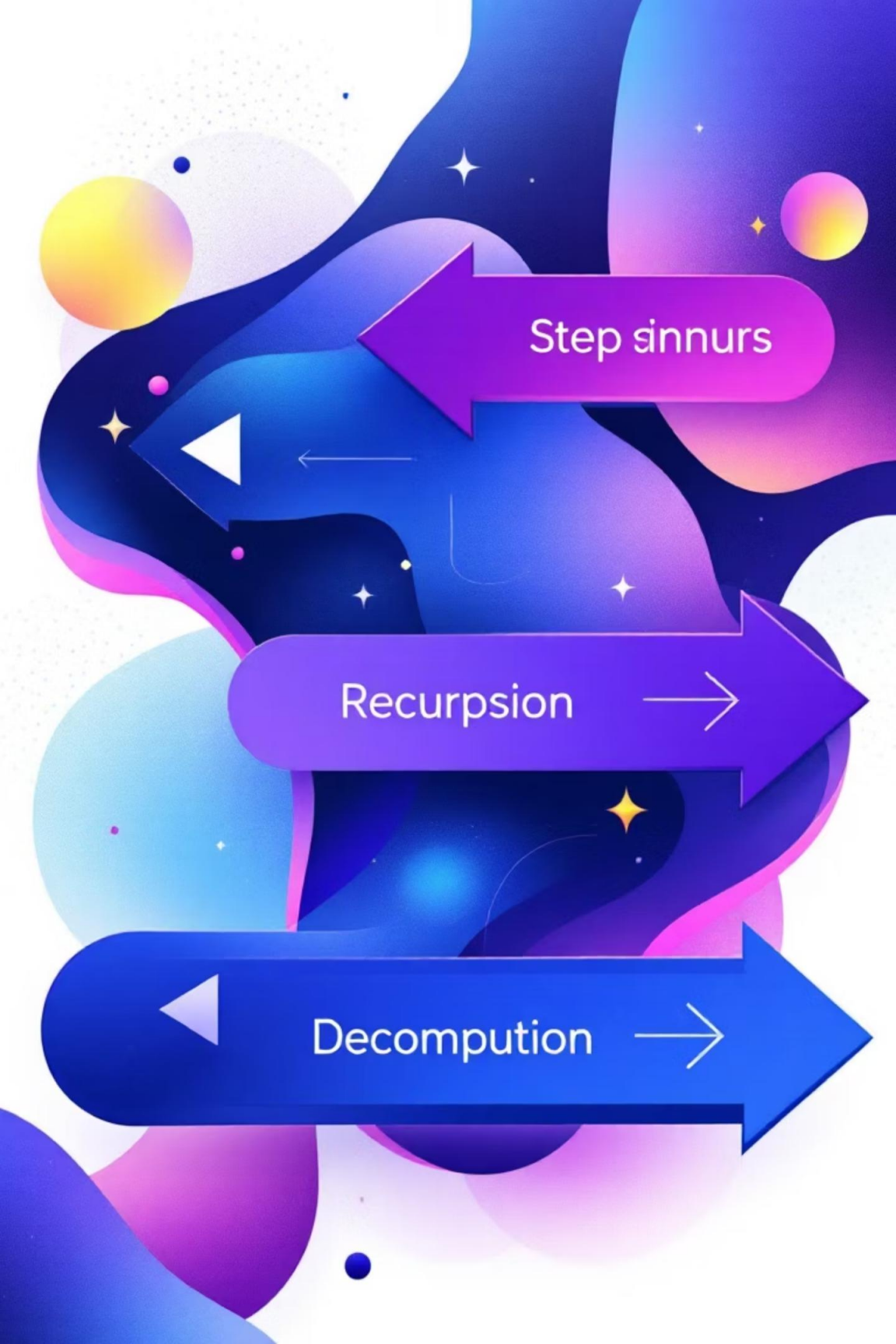
The Problem

Objective

Move all disks from peg A to peg C, following two rules:

1. Move one disk at a time
2. Never place a larger disk on a smaller one



An abstract graphic on the left side of the slide. It features several overlapping, wavy, organic shapes in shades of blue, purple, and pink. Three large, stylized arrows are integrated into the design. The top arrow is purple and points left, with the text 'Step sinners' inside. The middle arrow is purple and points right, with the text 'Recurpsion' inside. The bottom arrow is blue and points right, with the text 'Decomputation' inside. The background is white with some faint, scattered dots and small star-like shapes.

Step sinners

Recurpsion

Decomputation

Recursive Insight

To move n disks from source to destination using auxiliary peg:

Move $n-1$ disks

From source to auxiliary

Move largest disk

From source to destination

Move $n-1$ disks

From auxiliary to destination

Pseudocode

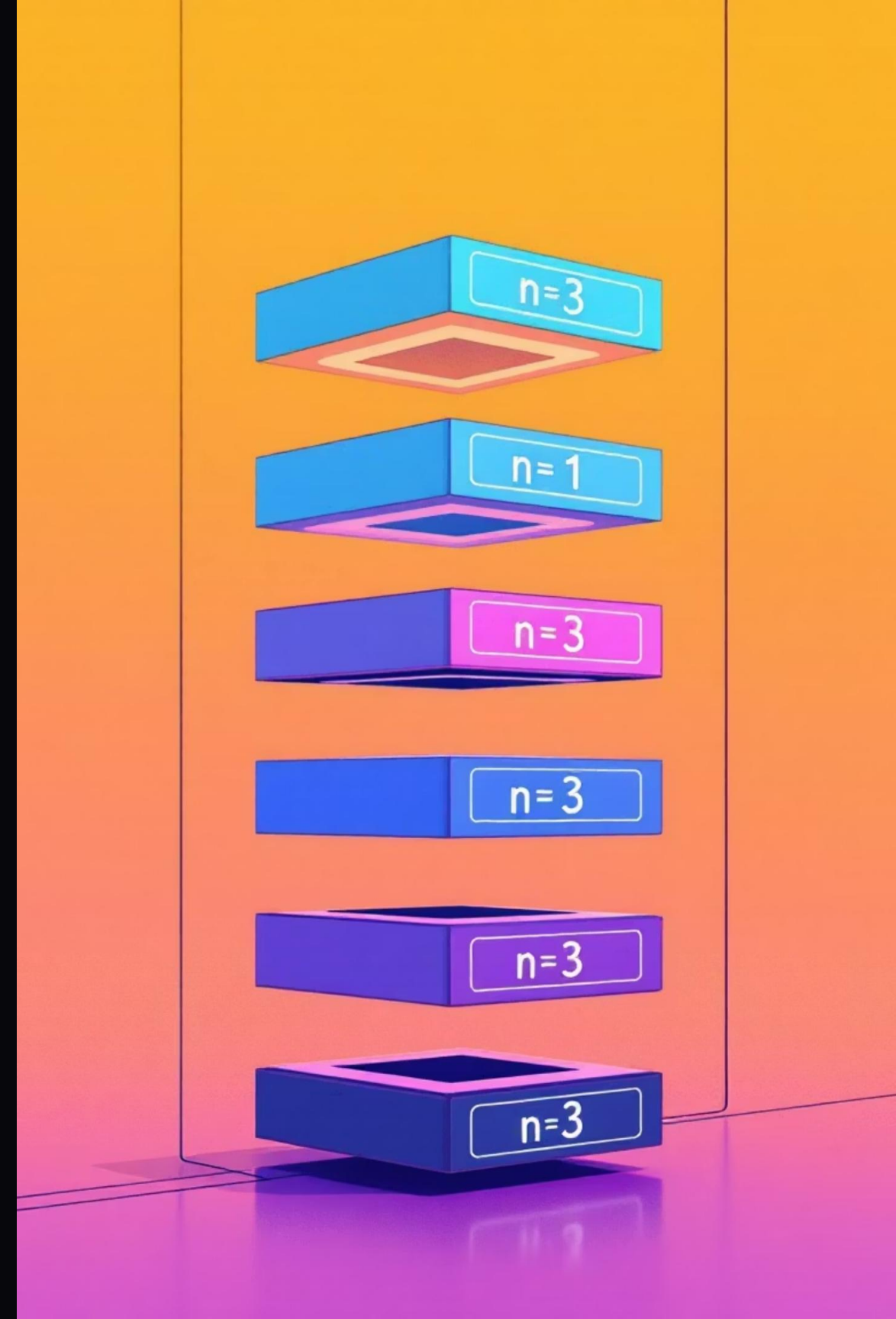
```
function hanoi(n, source, destination, auxiliary):    if n == 1:        move disk from source to destination
    else:        hanoi(n-1, source, auxiliary, destination)        move disk from source to destination
        hanoi(n-1, auxiliary, destination, source)
```

Base case: $n=1$ stops recursion. Recursive case: three function calls per invocation.

Call Stack Execution

For $n=3$ disks, the call stack grows and shrinks as recursion unfolds:

Each recursive call pushes a new frame. Base cases pop frames, returning control to parent calls.



Time Complexity Analysis

$$2^n - 1$$

Total Moves

For n disks, exactly $2^n - 1$ moves required.

$$O(2^n)$$

Time Complexity

Exponential growth; each disk doubles the moves.

$$O(n)$$

Space Complexity

Call stack depth equals n (maximum recursion depth).

Example: n=2 Disks

Step	Move	State A	State C
1	Small: A→B	[Large]	[]
2	Large: A→C	[]	[Large]
3	Small: B→C	[]	[Large, Small]

Three moves total: $2^2 - 1 = 3$. Demonstrates base case and recursive decomposition.

Why Recursion Works

Self-Similar Subproblems

Each recursive call solves an identical problem with smaller input ($n-1$).

Base Case Termination

When $n=1$, recursion stops, preventing infinite loops.

Call Stack Management

Stack frames preserve state; each level returns control correctly to its caller.

Key Takeaways

1 Recursion Elegance

Tower of Hanoi demonstrates how recursion simplifies complex problems through self-reference.

2 Exponential Complexity

$O(2^n)$ time complexity shows why recursive solutions can become impractical for large n .

3 Call Stack Insight

Understanding stack depth and frame management is essential for analysing recursive algorithms.